



POORNIMA
COLLEGE OF ENGINEERING

An autonomous institution approved by RTU, AICTE & UGC • NAAC A+ Accredited



Vulnerability Assessment and Penetration Testing Lab

Lab Manual (Course Code: 245CY4-23)

III Year - V Semester

B.Tech. Computer Science and Engineering (Cyber Security)

Department of Advance Computing (AI & Data Science)

Session: 2026-27

Poornima College of Engineering, Jaipur

ISI-6, RIICO Institutional Area, Sitapura, Jaipur-302022, Rajasthan

Phone/Fax: 0141-2770790-92 | www.pce.poornima.org

TABLE OF CONTENTS

TABLE OF CONTENTS	2
INSTITUTE VISION & MISSION	4
DEPARTMENT VISION & MISSION	4
RTU SYLLABUS AND MARKING SCHEME	5
EVALUATION SCHEME	5
DISTRIBUTION OF MARKS FOR EACH EXPERIMENT	5
LAB OUTCOMES AND ITS MAPPING WITH PO & PSO	6
LO-PO-PSO MAPPING MATRIX OF COURSE	6
PROGRAM OUTCOMES (POs)	7
PROGRAM SPECIFIC OUTCOMES (PSOs)	7
RUBRICS FOR LAB	9
LAB CONDUCTION PLAN	10
DISTRIBUTION OF LAB HOURS	10
LAB ROTOR PLAN	11
ROTOR-1	11
ROTOR-2	11
GENERAL LAB INSTRUCTIONS	12
LAB SPECIFIC SAFETY RULES	13
Before Entering the Lab	13
While Working in the Lab	13
Ethical and Legal Guidelines Specific to VAPT	13
ZERO LAB	14
Introduction to the Lab	14
Course Objectives	14
What is VAPT?	14
The Penetration Testing Lifecycle	14
Setting up the Lab Environment	14
Key Concepts	14
Ethical & Legal Considerations	15
Lab Expectations	15
EXPERIMENT - 1	16
EXPERIMENT - 2	18
EXPERIMENT - 3	20
EXPERIMENT - 4	21
EXPERIMENT - 5	23

EXPERIMENT - 6	25
EXPERIMENT - 7	27
EXPERIMENT - 8	29
EXPERIMENT - 9	31
EXPERIMENT - 10	33
EXPERIMENT - 11	35
EXPERIMENT - 12	37
EXPERIMENT - 13	39
EXPERIMENT - 14	41

INSTITUTE VISION & MISSION

VISION

To create a knowledge-based society with scientific temper, team spirit and dignity of labour to face the global competitive challenges.

MISSION

To evolve and develop skill-based systems for effective delivery of knowledge so as to equip young professionals with dedication & commitment to excellence in all spheres of life.

DEPARTMENT VISION & MISSION

VISION

To become the most preferred department for advanced computing and cyber security programs through creating an appropriate teaching-learning and skill-upgradation environment that fulfils current industry needs.

MISSION

1. To create an experiential learning environment that enables students to compete globally in the advanced computing and cyber security domains, and to contribute significantly to research and the discovery of new knowledge.
2. To adapt latest technological tools and contribute significantly to the advancement of knowledge in computing and security applications for industry, society and environment.
3. To inculcate essential characteristics in students for their all-round professional development, interaction with industry and society, and lifelong learning.
4. To create R&D infrastructure and centres of excellence in various advanced computing and cyber security sub-domains.

RTU SYLLABUS AND MARKING SCHEME

245CY4-23: VAPT LAB	
Credit: 1	Max. Marks: 100 (IA: 60, ETE: 40)
OL + OT + 2P	End Term Exam: 2 Hours

LIST OF EXPERIMENTS

S. No.	Name of Experiment
1	Clone a Live Website for Offline Browsing and Analysis using HTTrack
2	Inspect HTTP Headers, Request Types, and Redirection Status using cURL
3	Identify Open Ports and Services on a Target Machine using Nmap
4	Perform Vulnerability Scanning and Interpret Results using Nessus / OpenVAS
5	Detect Common Web Vulnerabilities (SQLi, XSS) using OWASP ZAP
6	Exploit a Known Vulnerability using Metasploit Framework
7	Detect Available Wi-Fi Networks and Analyze Security Configurations
8	Perform Password Brute Force on SSH or FTP
9	Identify and Exploit SQL Injection in a Test Environment
10	Create a Simulated Phishing Email and Track Which Users Respond
11	Assess and Report Physical Security Vulnerabilities in the Lab/Office
12	Conduct a Mock Test to Find Sensitive Data from Discarded Documents (Safe Simulation)
13	Capture and Analyze Unencrypted Network Traffic
14	Use SSL Labs or OpenSSL to Test the Security of a Web Server's SSL Configuration

EVALUATION SCHEME

I+II Mid Term (Experiment)	I+II Mid Term (Viva)	Attendance & Performance	End Term (Experiment)	End Term (Viva)	Total Marks
30	10	40	30	10	100

DISTRIBUTION OF MARKS FOR EACH EXPERIMENT

Attendance	Record	Performance	Total
2	3	5	10

LAB OUTCOMES AND ITS MAPPING WITH PO & PSO

LAB OUTCOMES

After completion of this course, students will be able to –

LO Code	Lab Outcome
245CY4-23.1	Set up and utilise reconnaissance and information-gathering tools (Kali Linux, HTTrack, cURL, Nmap) to profile the attack surface of target systems and web assets.
245CY4-23.2	Perform vulnerability assessment of networks and web applications using automated scanners (Nessus/OpenVAS, OWASP ZAP) and correctly interpret and prioritise the reported findings.
245CY4-23.3	Exploit identified vulnerabilities in a controlled lab environment using tools such as Metasploit, sqlmap, and Hydra, and evaluate wireless network and unencrypted traffic security.
245CY4-23.4	Assess social-engineering, physical-security, and cryptographic (SSL/TLS) risks, and prepare a professional penetration-testing report with remediation recommendations, in accordance with ethical and legal guidelines.

LO-PO-PSO MAPPING MATRIX OF COURSE

LO/PO/PSO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
245CY4-23.1	2	2	-	-	2	-	-	-	-	-	-	-	3	1	-
245CY4-23.2	1	3	2	2	3	-	-	-	-	-	-	1	3	2	-
245CY4-23.3	1	2	3	2	3	-	-	1	1	-	-	1	3	2	1
245CY4-23.4	-	2	2	1	2	1	-	3	2	2	1	2	2	3	2

PROGRAM OUTCOMES (POs)

Code	Description
PO1	Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
PO5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	The engineer and society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.
PO7	Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for, sustainable development.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO9	Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, including writing effective reports and design documentation, making effective presentations, and giving and receiving clear instructions.
PO11	Project management and finance: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects in multidisciplinary environments.
PO12	Life-long learning: Recognize the need for, and have the preparation and ability to engage in, independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

Code	Description
PSO1	The ability to understand and apply knowledge of mathematics, system analysis & design, data modelling, cloud technology, and latest tools to develop computer-based solutions in the areas of system software, multimedia, web applications, big data analytics, IoT, business intelligence, networking and cyber security systems.

Code	Description
PSO2	The ability to understand the evolutionary changes in computing, apply standards and ethical practices in project development using latest tools & technologies to solve societal problems and meet the challenges of the future.
PSO3	The ability to employ modern computing and security tools and platforms to be an entrepreneur, and to pursue lifelong learning and higher studies.

RUBRICS FOR LAB

Laboratory Evaluation Rubrics:

Criteria	Outstanding (>90%)	Admirable (70-90%)	Average (40-69%)	Inadequate (<40%)
Procedure & Format	All steps followed correctly, output/input variables correctly identified	Most steps followed correctly	Some steps followed, minor gaps in variables	Procedure not followed / variables not defined
Individual/Team Work	Excellent coordination and individual contribution	Good coordination among group members	Average participation	Very poor / no participation
Data Precision & Collection	Data correct in size and collected from proper sources	Data collected is appropriate but not from best sources	Data not sufficiently appropriate	Data neither appropriate nor from proper sources
Timing / Punctuality	Record & performance submitted on the same day	Submitted by next turn	Submitted before deadline (with delay)	Submitted after deadline
Data Analysis & Results	Results exhaustively analyzed, all diagrams/tables prepared neatly	Results analyzed but not all variables/diagrams covered	Results included but not as per experimental scenario	No proper analysis/diagrams included

LAB CONDUCTION PLAN

Total number of Experiments: 14 Total number of turns required: 15 (including Zero Lab)

Experiment Number	Scheduled Week
Zero Lab (Orientation & Environment Setup)	Week 1
Experiment - 1	Week 2
Experiment - 2	Week 3
Experiment - 3	Week 4
Experiment - 4	Week 5
Experiment - 5	Week 6
I Mid Term	Week 7
Experiment - 6	Week 8
Experiment - 7	Week 9
Experiment - 8	Week 9
Experiment - 9	Week 10
Experiment - 10	Week 10
Experiment - 11	Week 11
Experiment - 12	Week 11
Experiment - 13	Week 12
Experiment - 14	Week 12
II Mid Term	Week 13

DISTRIBUTION OF LAB HOURS

S. No.	Activity	Time (120 minutes)
1	Attendance	5
2	Explanation of Experiment & Logic	25
3	Performing the Experiment	50
4	File Checking	15
5	Viva/Quiz	15
6	Solving of Queries	10

LAB ROTOR PLAN

ROTOR-1

Ex. No.	Name of Experiment
1	Clone a Live Website for Offline Browsing and Analysis using HTTrack
2	Inspect HTTP Headers, Request Types, and Redirection Status using cURL
3	Identify Open Ports and Services on a Target Machine using Nmap
4	Perform Vulnerability Scanning and Interpret Results using Nessus / OpenVAS
5	Detect Common Web Vulnerabilities (SQLi, XSS) using OWASP ZAP
6	Exploit a Known Vulnerability using Metasploit Framework
7	Detect Available Wi-Fi Networks and Analyze Security Configurations

ROTOR-2

Ex. No.	Name of Experiment
8	Perform Password Brute Force on SSH or FTP
9	Identify and Exploit SQL Injection in a Test Environment
10	Create a Simulated Phishing Email and Track Which Users Respond
11	Assess and Report Physical Security Vulnerabilities in the Lab/Office
12	Conduct a Mock Test to Find Sensitive Data from Discarded Documents (Safe Simulation)
13	Capture and Analyze Unencrypted Network Traffic
14	Use SSL Labs or OpenSSL to Test the Security of a Web Server's SSL Configuration

GENERAL LAB INSTRUCTIONS

DO'S

1. Enter the lab on time and leave at the proper time.
2. Wait for the previous class to leave before the next class enters.
3. Keep bags outside in the respective racks.
4. Utilize lab hours for the corresponding experiment only.
5. Shut down the machine before leaving the lab unless a member of lab staff specifically instructs otherwise.
6. Leave the lab at least as clean and orderly as you found it.
7. Report any equipment/room problem (system not responding, cooling, lighting, etc.) to lab staff immediately; do not attempt to fix it yourself.

DON'TS

1. Don't abuse the equipment.
2. Do not adjust the air-conditioning; inform lab staff if the temperature is uncomfortable.
3. Do not attempt to reboot a computer yourself; report problems to lab staff.
4. Do not remove or modify any software/file without permission.
5. Do not disconnect network equipment without explicit permission from lab staff.
6. Don't monopolize a machine; log out if leaving for more than 10-15 minutes.
7. Don't use the internet or chat applications outside the scope of the scheduled experiment.
8. Do not download/upload media files unrelated to the lab work.
9. No games are allowed during lab sessions.
10. No external hardware/USB drives may be connected without prior permission of the lab in-charge.
11. No food or drink is allowed in the lab or near equipment.
12. Don't bring external material into the lab except your lab record, copy, and books.
13. Mobile phones must be kept in silent mode; avoid use unless necessary.
14. Keep noise levels down and be considerate of others.
15. For any doubts, approach the faculty or lab in-charge for support.

VAPT LAB SPECIFIC CONDUCT

- All scanning, exploitation, and cracking activities in this lab must be performed only against machines/VMs provided within the isolated lab network — never against any live, production, or third-party system.
- Unauthorised access to, or scanning of, any computer system or network outside this lab is a punishable offence under the Information Technology Act, 2000.
- Every experiment must be logged in the lab record along with the target system used, tools used, and observed results, for accountability.

LAB SPECIFIC SAFETY RULES

Before Entering the Lab

1. All students must prepare the theory regarding the next experiment/tool in advance.
2. Students must bring their lab records as per the lab schedule.
3. The previous experiment must be written up in the lab record.
4. Any screenshots/outputs must be pasted/printed and labelled properly in the lab record.
5. All students must follow lab instructions, failing which they may not be allowed to attend the lab session.

While Working in the Lab

1. Adhere to the experimental schedule as instructed by the lab in-charge/faculty.
2. Get the previously performed experiment signed by the faculty/lab in-charge.
3. Get the output of the current experiment verified by the faculty/lab in-charge.
4. Each student should work only on their assigned virtual machine/system at each turn.
5. Take responsibility for valuable accessories and lab equipment.

Ethical and Legal Guidelines Specific to VAPT

- All penetration-testing activities must be confined to the isolated lab network and pre-approved target VMs (e.g., Metasploitable2, DVWA) provided by the department.
- Written/verbal authorisation from the faculty in-charge is mandatory before performing any scanning, exploitation, brute-force, or sniffing exercise.
- Students must never use the tools and techniques learnt in this lab against any system, network, or individual outside the authorised lab environment.
- Any tools or scripts capable of causing denial-of-service must be used with caution and only as instructed.
- Violation of these guidelines may attract institutional disciplinary action in addition to potential liability under the Information Technology Act, 2000.

ZERO LAB

Introduction to the Lab

Welcome to the Vulnerability Assessment and Penetration Testing (VAPT) Lab. This course introduces the systematic process of identifying, analysing, and safely demonstrating security weaknesses in networks, web applications, wireless systems, and physical/human processes, using industry-standard open-source tools within a strictly controlled and authorised lab environment.

Course Objectives

- Understand the phases of a penetration test: reconnaissance, scanning, vulnerability assessment, exploitation, and reporting.
- Gain hands-on experience with industry-standard tools such as HTTrack, cURL, Nmap, Nessus/OpenVAS, OWASP ZAP, Metasploit, aircrack-ng, Hydra, sqlmap, Wireshark, and OpenSSL.
- Learn to assess technical (network, web, wireless, cryptographic) as well as human and physical security risks.
- Develop the ability to document findings professionally and recommend appropriate remediation.
- Understand the ethical, legal, and professional responsibilities of a penetration tester.

What is VAPT?

Vulnerability Assessment (VA) is the process of identifying and classifying security weaknesses in a system, typically using automated scanners. Penetration Testing (PT) goes a step further by actively attempting to exploit identified vulnerabilities in a controlled manner to demonstrate real-world impact. Together, VAPT provides a comprehensive picture of an organisation's security posture, combining breadth (VA) with depth of validation (PT).

The Penetration Testing Lifecycle

1. Planning & Reconnaissance – defining scope, gathering information about the target (passive and active).
2. Scanning – identifying live hosts, open ports, running services (e.g., using Nmap).
3. Vulnerability Assessment – mapping discovered services to known vulnerabilities (e.g., using Nessus/OpenVAS, OWASP ZAP).
4. Exploitation – validating vulnerabilities by safely exploiting them (e.g., using Metasploit, sqlmap, Hydra) in an authorised environment.
5. Post-Exploitation & Reporting – documenting access gained, evidence collected, and preparing a report with risk ratings and remediation.

Setting up the Lab Environment

- Attacker machine: Kali Linux (VM), which comes pre-loaded with most tools used in this course.
- Target machine(s): deliberately vulnerable VMs such as Metasploitable2, DVWA, or bWAPP, used strictly for training.
- Network: both attacker and target VMs must be placed on an isolated host-only/NAT virtual network with no route to the internet or the institute's production network.
- Virtualization platform: VirtualBox / VMware Workstation to host the Kali and target VMs.

Key Concepts

- Threat, Vulnerability, and Risk: a threat is a potential cause of harm; a vulnerability is a weakness that can be exploited; risk is the likelihood and impact of a threat exploiting a vulnerability.
- CIA Triad: Confidentiality, Integrity, and Availability — the three core objectives that security controls aim to protect.
- Types of Testing: Black-box (no prior knowledge), White-box (full knowledge), Grey-box (partial knowledge) testing.
- CVE/CVSS: Common Vulnerabilities and Exposures identifiers and the Common Vulnerability Scoring System used to rate severity.

Ethical & Legal Considerations

- Always obtain explicit written authorisation before testing any system — in this course, that authorisation is scoped strictly to the lab's isolated VMs.
- Never use the tools and techniques taught in this lab against systems, networks, or individuals outside the authorised lab environment.
- Unauthorised access to a computer system or network is an offence under the Information Technology Act, 2000, and applicable data protection law.
- Maintain confidentiality of any findings and follow responsible-disclosure principles.

Lab Expectations

- Ensure the Kali Linux and target VMs are set up and functioning by the next session.
- Maintain a lab record documenting the objective, tools, procedure, output, and viva preparation for every experiment.
- Each experiment will conclude with a short viva to reinforce conceptual understanding.

EXPERIMENT - 1

Clone a Live Website for Offline Browsing and Analysis using HTTrack

OBJECTIVE

To clone a live website using the HTTrack Website Copier tool for offline browsing, and to analyze the downloaded site structure for reconnaissance purposes.

TOOLS / SOFTWARE REQUIRED

- Kali Linux
- HTTrack Website Copier (GUI: webhttrack / CLI: httrack)
- A test website (e.g., a lab-hosted DVWA/local site or a website you are authorised to mirror)

THEORY

Website mirroring is an important reconnaissance step in the information-gathering phase of a penetration test. HTTrack downloads an entire website (HTML pages, images, CSS, JavaScript) to a local directory, recreating the original site's relative link structure. Analysts use offline copies to study site structure, hidden comments, form actions, and client-side scripts without repeatedly hitting the live server, reducing the chance of detection and load on the target.

PROCEDURE

1. Install HTTrack on Kali Linux: `sudo apt update && sudo apt install httrack webhttrack`.
2. Launch the GUI wizard with the command `webhttrack`, or use the CLI directly.
3. Provide a project name and base directory to store the mirrored site.
4. Enter the target URL of the website you are authorised to test.
5. Configure mirroring options such as depth of links to follow, connection limits, and file types to exclude (e.g., large media files).
6. Start the mirroring process and monitor the transfer log for errors (403/404 responses, robots.txt restrictions).
7. Open the downloaded `index.html` locally in a browser and browse the mirrored site offline.
8. Inspect the local file structure for directory listing, hidden pages, comments in HTML/JS source, and form endpoints that may reveal backend technology.

SAMPLE COMMANDS / OUTPUT

```
CLI equivalent:  
httrack "http://testsite.local" -O "/root/mirror/testsite" "+*.testsite.local/*" -v
```

PRECAUTIONS

- Only mirror websites you own or are explicitly authorised to test — cloning third-party sites without permission may violate the IT Act, 2000 and copyright law.
- Respect robots.txt and rate-limit requests to avoid overloading the target server.
- Do not publish or redistribute the cloned content.

VIVA QUESTIONS

1. What is website mirroring and why is it used in the reconnaissance phase of penetration testing?
2. What is the difference between passive and active reconnaissance? Where does HTTrack fit in?
3. What information can an attacker gain by studying a mirrored copy of a website?

4. What precautions should be taken while mirroring a live website?
5. Name two alternatives to HTTrack for website mirroring.

EXPERIMENT - 2

Inspect HTTP Headers, Request Types, and Redirection Status using cURL

OBJECTIVE

To use cURL on Kali Linux to inspect HTTP request/response headers, identify the HTTP methods supported by a server, and trace redirection chains of a website.

TOOLS / SOFTWARE REQUIRED

- Kali Linux
- cURL

THEORY

cURL is a command-line tool for transferring data using various protocols, most commonly HTTP/HTTPS. During reconnaissance, HTTP response headers reveal the web server software and version, cookies, security headers (or the lack of them, such as missing X-Frame-Options or Content-Security-Policy), and caching behaviour. Observing redirection chains (3xx status codes) helps map how a site funnels traffic and can reveal internal hostnames.

PROCEDURE

1. Open a terminal on Kali Linux (cURL is installed by default).
2. Fetch only the response headers of a target URL: `curl -I http://target.local`.
3. Fetch full verbose output including the request and response headers: `curl -v http://target.local`.
4. Test which HTTP methods the server accepts using the OPTIONS method: `curl -X OPTIONS -i http://target.local`.
5. Follow and display a redirection chain using the -L (location) flag: `curl -IL http://target.local`.
6. Examine response headers such as Server, X-Powered-By, Set-Cookie, and security headers to fingerprint the technology stack.
7. Record all HTTP status codes returned during the redirection trace (301, 302, 200) and the final destination URL.

SAMPLE COMMANDS / OUTPUT

```
curl -I https://target.local
curl -v https://target.local
curl -X OPTIONS -i https://target.local
curl -IL http://target.local
```

PRECAUTIONS

- Only send requests to systems you are authorised to test.
- Avoid flooding the server with repeated requests (do not use this as a load-testing tool).
- Note that some headers may be spoofed by the server, so treat fingerprinting results as indicative, not conclusive.

VIVA QUESTIONS

1. What is the difference between `curl -I` and `curl -v`?
2. Why are HTTP security headers (CSP, X-Frame-Options, HSTS) important, and how would their absence be flagged in a report?
3. What does a 301 vs a 302 redirect indicate, and why does the distinction matter to a penetration tester?

4. How can the Server and X-Powered-By headers be used for fingerprinting, and how can they be spoofed to mislead an attacker?
5. What is the OPTIONS HTTP method used for during reconnaissance?

EXPERIMENT - 3

Identify Open Ports and Services on a Target Machine using Nmap

OBJECTIVE

To perform network scanning on a target machine using Nmap in order to identify open ports, running services, and their versions.

TOOLS / SOFTWARE REQUIRED

- Kali Linux
- Nmap
- A lab target VM (e.g., Metasploitable2) on an isolated network

THEORY

Nmap (Network Mapper) is the industry-standard tool for host discovery and port scanning. It works by sending crafted packets (TCP SYN, TCP connect, UDP, etc.) to a target and analysing the responses to determine which ports are open, closed, or filtered. Combined with service/version detection (-sV) and OS fingerprinting (-O), Nmap builds a profile of the attack surface that guides the subsequent vulnerability-assessment phase.

PROCEDURE

1. Ensure both the attacker (Kali) and target VM are on the same isolated lab network.
2. Identify the target's IP address, e.g., using netdiscover or arp-scan on the lab subnet.
3. Perform a basic TCP SYN scan of the top 1000 ports: `nmap -sS <target-ip>`.
4. Perform a full port range scan: `nmap -p- <target-ip>`.
5. Enable service and version detection: `nmap -sV <target-ip>`.
6. Enable OS detection (requires root privileges): `sudo nmap -O <target-ip>`.
7. Combine common flags for a comprehensive scan: `nmap -sS -sV -O -p- <target-ip> -oN scan_results.txt`.
8. Analyse the output to list each open port, its service, version, and note any outdated/insecure services (e.g., Telnet, FTP with anonymous login).

SAMPLE COMMANDS / OUTPUT

```
sudo nmap -sS -sV -O -p- 192.168.56.101 -oN scan_results.txt
```

PRECAUTIONS

- Only scan machines within the authorised lab network — scanning external/live systems without permission is illegal under the IT Act, 2000.
- Aggressive scans can be noisy and may trigger IDS/IPS alerts; use scan timing options (-T) responsibly.
- Save scan output for later comparison and reporting.

VIVA QUESTIONS

1. What is the difference between a TCP SYN scan (-sS) and a TCP connect scan (-sT)?
2. What do the port states open, closed, and filtered mean in Nmap output?
3. Why is service/version detection (-sV) important for vulnerability assessment?
4. What are the risks of running an aggressive Nmap scan against a production network?
5. How does OS fingerprinting work in Nmap?

EXPERIMENT - 4

Perform Vulnerability Scanning and Interpret Results using Nessus / OpenVAS

OBJECTIVE

To configure and run an automated vulnerability scan against a target machine using Nessus (or OpenVAS if Nessus is unavailable) and to interpret the severity and remediation guidance of the reported vulnerabilities.

TOOLS / SOFTWARE REQUIRED

- Kali Linux
- Nessus Essentials or OpenVAS (GVM)
- A lab target VM (e.g., Metasploitable2)

THEORY

Vulnerability scanners compare the services, versions, and configurations discovered on a target against databases of known vulnerabilities (CVE, NVD) and misconfiguration checks. Nessus and OpenVAS both provide plugin/NVT-based scanning engines that classify findings by severity (Critical/High/Medium/Low/Info) using CVSS scores, and provide remediation references, letting the tester prioritise which weaknesses pose the greatest risk.

PROCEDURE

1. Install and activate Nessus Essentials (or install OpenVAS/GVM using `sudo apt install openvas / gvm-setup`).
2. Access the scanner's web interface (<https://localhost:8834> for Nessus, <https://localhost:9392> for OpenVAS).
3. Create a new scan and specify the target IP address of the lab VM.
4. Select a scan policy/template such as 'Basic Network Scan'.
5. Launch the scan and wait for it to complete.
6. Review the results dashboard, sorted by severity.
7. For each significant finding, note the CVE ID/plugin ID, CVSS score, affected service, and the recommended remediation.
8. Export the scan report (PDF/HTML) for inclusion in the penetration-testing report.

SAMPLE COMMANDS / OUTPUT

```
Example finding to record:  
Host: 192.168.56.101 | Port: 21/tcp | Plugin: vsftpd 2.3.4 Backdoor | Severity: Critical  
| CVSS: 10.0 | Remediation: Upgrade vsftpd and restrict FTP access.
```

PRECAUTIONS

- Run scans only against the isolated lab target — network-wide scans can disrupt production services.
- Some scan checks (denial-of-service plugins) can crash vulnerable services; disable such plugins on shared lab networks.
- Keep the vulnerability feed/plugin database updated for accurate results.

VIVA QUESTIONS

1. What is the difference between vulnerability scanning and penetration testing?
2. How is CVSS score used to prioritise remediation?

3. What is a false positive in vulnerability scanning, and how would you verify one?
4. Compare Nessus and OpenVAS in terms of licensing and plugin coverage.
5. What information should a good vulnerability report contain for each finding?

EXPERIMENT - 5

Detect Common Web Vulnerabilities (SQLi, XSS) using OWASP ZAP

OBJECTIVE

To use OWASP ZAP (Zed Attack Proxy) to perform automated and manual scanning of a web application to detect common vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS).

TOOLS / SOFTWARE REQUIRED

- Kali Linux
- OWASP ZAP
- A vulnerable web application (e.g., DVWA / OWASP Juice Shop / bWAPP hosted locally)

THEORY

OWASP ZAP is a free, open-source web application security scanner. It intercepts traffic between the browser and the target application as a proxy, allowing manual inspection/modification of requests, and also runs automated spidering and active-scan engines that inject test payloads to detect vulnerabilities listed in the OWASP Top 10, notably SQL Injection (unsanitised database queries) and Cross-Site Scripting (unsanitised output allowing script injection).

PROCEDURE

1. Launch OWASP ZAP on Kali Linux and configure the browser (or use ZAP's built-in browser) to proxy through ZAP (default 127.0.0.1:8080).
2. Enter the target URL of the vulnerable web application in the 'Quick Start' Automated Scan tab.
3. Run the Spider to crawl and discover all reachable pages, forms, and parameters.
4. Run the Active Scan to automatically inject test payloads (SQLi, XSS, etc.) into discovered parameters.
5. Review the Alerts tab, which lists findings grouped by risk (High/Medium/Low/Informational).
6. For each SQL Injection or XSS alert, inspect the request/response pair to understand the vulnerable parameter and payload used.
7. Manually verify a sample finding, e.g., submit a payload such as ' OR '1'='1 in a login form to confirm SQL Injection, or <script>alert(1)</script> in an input field to confirm reflected XSS.
8. Export the ZAP report (HTML/PDF) summarising all detected vulnerabilities.

SAMPLE COMMANDS / OUTPUT

```
Manual verification payloads (used only on the authorised lab target):  
SQLi test: ' OR '1'='1  
XSS test: <script>alert('XSS-Test')</script>
```

PRECAUTIONS

- Only run active scans against applications you are authorised to test — active scanning sends attack payloads that can corrupt data.
- Take a backup/snapshot of the target VM before active scanning.
- Do not use these payloads on any production or third-party website.

VIVA QUESTIONS

1. What is the difference between passive scanning and active scanning in OWASP ZAP?
2. Explain how SQL Injection works and how it can be prevented (e.g., parameterised queries).

3. What is the difference between reflected, stored, and DOM-based XSS?
4. What is the OWASP Top 10, and why is it referenced in web application security testing?
5. How does ZAP's spidering function help in discovering the attack surface of a web application?

EXPERIMENT - 6

Exploit a Known Vulnerability using Metasploit Framework

OBJECTIVE

To use the Metasploit Framework to identify and exploit a known vulnerability on a deliberately vulnerable lab target, and to gain a basic understanding of exploit/payload usage.

TOOLS / SOFTWARE REQUIRED

- Kali Linux
- Metasploit Framework (msfconsole)
- Metasploitable2 (or similar vulnerable VM) on an isolated network

THEORY

Metasploit Framework is a modular penetration-testing platform containing a large database of exploits, payloads, and auxiliary modules. Given a known vulnerable service (identified in earlier experiments via Nmap/Nessus, such as the vsftpd 2.3.4 backdoor on Metasploitable2), Metasploit lets a tester select a matching exploit module, configure a payload (e.g., a reverse shell), and execute it to demonstrate real-world impact, which is essential for proving risk rather than only theoretical severity.

PROCEDURE

1. Start the PostgreSQL database and msfconsole: `sudo systemctl start postgresql && msfconsole`.
2. Search for an exploit module matching a known vulnerable service, e.g.: `search vsftpd`.
3. Select the module: use `exploit/unix/ftp/vsftpd_234_backdoor`.
4. View and set the required options: `show options`, then set `RHOSTS <target-ip>`.
5. Select an appropriate payload if applicable: set `PAYLOAD cmd/unix/interact`.
6. Execute the exploit: `run` (or `exploit`).
7. On successful exploitation, verify shell access on the target by running basic commands (`whoami`, `uname -a`).
8. Document the exploited service, CVE reference, and the evidence of access obtained.
9. Exit the session cleanly and record findings for the report.

SAMPLE COMMANDS / OUTPUT

```
msf6 > search vsftpd
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(vsftpd_234_backdoor) > set RHOSTS 192.168.56.101
msf6 exploit(vsftpd_234_backdoor) > run
```

PRECAUTIONS

- Exploitation must be performed strictly against the isolated lab VM — running exploits against unauthorised systems is a criminal offence under the IT Act, 2000.
- Keep the attacker and target machines on a host-only/NAT network with no route to the internet or institute production network.
- Take VM snapshots before exploitation so the target can be reverted after the exercise.

VIVA QUESTIONS

1. What are the components of Metasploit — exploit, payload, and auxiliary module — and how do they relate?

2. What is the difference between a bind shell and a reverse shell payload?
3. Why is it important to verify that an exploit matches the exact version of the target service?
4. What legal and ethical safeguards must be in place before using Metasploit outside a lab environment?
5. What is the role of msfvenom in the Metasploit ecosystem?

EXPERIMENT - 7

Detect Available Wi-Fi Networks and Analyze Security Configurations

OBJECTIVE

To scan for nearby Wi-Fi networks using wireless auditing tools and analyse their security configuration (open, WEP, WPA/WPA2/WPA3) to identify weak wireless setups.

TOOLS / SOFTWARE REQUIRED

- Kali Linux
- A wireless network adapter supporting monitor mode
- aircrack-ng suite (airmon-ng, airodump-ng)

THEORY

Wireless networks broadcast beacon frames that reveal the SSID, channel, signal strength, and the encryption/authentication scheme in use. Weak configurations such as open networks, WEP, or WPA with a weak pre-shared key are highly susceptible to eavesdropping and unauthorised access. Wireless auditing tools such as the aircrack-ng suite allow a tester to place the wireless adapter into monitor mode and passively capture this beacon information without associating to any network.

PROCEDURE

1. Connect a compatible wireless adapter and identify its interface name: iwconfig or ip link.
2. Enable monitor mode on the adapter: `sudo airmon-ng start wlan0` (creates wlan0mon).
3. Scan for nearby access points: `sudo airodump-ng wlan0mon`.
4. Record each visible SSID, BSSID, channel, signal strength, and the ENC/CIPHER/AUTH columns (e.g., WPA2 CCMP PSK, WEP, OPN for open).
5. Classify each network's security level: Open (no encryption), WEP (deprecated/broken), WPA/WPA2-Personal (PSK), WPA2/WPA3-Enterprise.
6. Note any networks broadcasting default or predictable SSIDs, which may indicate default router configurations.
7. Stop monitor mode when done: `sudo airmon-ng stop wlan0mon`.

SAMPLE COMMANDS / OUTPUT

```
sudo airmon-ng start wlan0
sudo airodump-ng wlan0mon
```

PRECAUTIONS

- This experiment only covers passive scanning/monitoring of visible networks — do not attempt to associate with, deauthenticate clients on, or crack the key of any network you do not own or have written authorisation to test.
- Unauthorised access to wireless networks is an offence under the IT Act, 2000.
- Conduct the scan only within the lab premises using the institute's own test access point.

VIVA QUESTIONS

1. What is the difference between WEP, WPA, WPA2, and WPA3 in terms of security?
2. Why is monitor mode required for wireless auditing, and how does it differ from normal (managed) mode?

3. What information can be gathered from a Wi-Fi beacon frame?
4. What makes an open Wi-Fi network risky, even without a key requirement?
5. What is a rogue access point / evil twin attack?

EXPERIMENT - 8

Perform Password Brute Force on SSH or FTP

OBJECTIVE

To perform a dictionary/brute-force attack against an SSH or FTP service on a lab target to demonstrate the risk of weak credentials, using a password-cracking tool such as Hydra.

TOOLS / SOFTWARE REQUIRED

- Kali Linux
- THC-Hydra
- A lab target VM running SSH/FTP with a weak test account
- A wordlist (e.g., rockyou.txt)

THEORY

Brute-force and dictionary attacks attempt many username/password combinations against an authentication service until a valid credential is found. Services such as SSH and FTP are common targets because they provide direct administrative or file access. Hydra is a fast, parallelised login-cracker supporting many protocols. This experiment demonstrates why weak, default, or reused passwords represent a significant attack surface and why account lockout, rate limiting, and key-based authentication are recommended defences.

PROCEDURE

1. Confirm the target service is reachable: `nmap -p 22,21 <target-ip>`.
2. Prepare (or use an existing) small wordlist of candidate passwords for the lab exercise.
3. Run Hydra against SSH with a known test username and the wordlist: `hydra -l <username> -P wordlist.txt ssh://<target-ip>`.
4. Alternatively, run Hydra against FTP: `hydra -l <username> -P wordlist.txt ftp://<target-ip>`.
5. Observe Hydra's output as it reports valid credential(s) found.
6. Log in using the discovered credentials to confirm access was obtained.
7. Record the time taken and number of attempts, and note this evidence in the report as justification for enforcing strong password policies and account-lockout thresholds.

SAMPLE COMMANDS / OUTPUT

```
hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.101
hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ftp://192.168.56.101
```

PRECAUTIONS

- Perform brute-force attacks only against a lab-created test account on an isolated target — never against real user accounts or production/live systems.
- Brute forcing can lock out accounts or trigger IDS alerts; coordinate lab timing to avoid disrupting shared infrastructure.
- Use a reasonably small/controlled wordlist in the lab session to keep the exercise within the allotted time.

VIVA QUESTIONS

1. What is the difference between a brute-force attack and a dictionary attack?

2. What countermeasures can mitigate password brute-forcing (e.g., account lockout, rate limiting, MFA, key-based auth)?
3. Why are SSH and FTP common targets for credential attacks?
4. How does Hydra parallelise login attempts, and how does this affect detection?
5. What is credential stuffing, and how does it differ from brute forcing?

EXPERIMENT - 9

Identify and Exploit SQL Injection in a Test Environment

OBJECTIVE

To manually identify and exploit a SQL Injection vulnerability in a deliberately vulnerable web application to extract data from the backend database, and to understand mitigation techniques.

TOOLS / SOFTWARE REQUIRED

- Kali Linux
- Web browser / Burp Suite (optional)
- sqlmap
- DVWA or bWAPP configured with low security

THEORY

SQL Injection occurs when user-supplied input is concatenated directly into a SQL query without proper sanitisation or parameterisation, allowing an attacker to alter the query's logic. This can be used to bypass authentication, extract data, or modify/delete records. Automated tools such as sqlmap can detect and exploit SQLi automatically by fuzzing parameters and enumerating databases, tables, and columns once a vulnerable parameter is confirmed.

PROCEDURE

1. Open DVWA (set security level to 'low') and navigate to the SQL Injection module.
2. Manually test the input field with a classic payload, e.g., ' OR '1'='1 to observe whether the query logic is bypassed.
3. Capture the vulnerable request URL/parameter (e.g., ?id=1).
4. Use sqlmap to confirm and automate exploitation: `sqlmap -u "http://target/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=...; security=low"`.
5. Enumerate available databases: `sqlmap -u "..." --cookie="..." --dbs`.
6. Enumerate tables within a chosen database: `sqlmap -u "..." --cookie="..." -D dvwa --tables`.
7. Dump data from a sensitive table (e.g., users) to demonstrate impact: `sqlmap -u "..." --cookie="..." -D dvwa -T users --dump`.
8. Document the vulnerable parameter, extracted data, and recommend mitigations such as parameterised queries/prepared statements, input validation, and least-privilege database accounts.

SAMPLE COMMANDS / OUTPUT

```
sqlmap -u "http://127.0.0.1/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --  
cookie="PHPSESSID=xxxx; security=low" --dbs
```

PRECAUTIONS

- Perform SQL Injection testing only on the intentionally vulnerable lab application (DVWA/bWAPP), never on live or third-party systems.
- Do not use extracted data for any purpose beyond documenting the vulnerability in the report.
- Restore the lab database to its original state after the exercise.

VIVA QUESTIONS

1. What is SQL Injection, and what are its common types (in-band, blind, out-of-band)?

2. How do prepared statements/parameterised queries prevent SQL Injection?
3. What is the purpose of the ' OR '1'='1 payload?
4. What is blind SQL injection, and how does sqlmap detect it?
5. What database privileges should a web application account have to limit SQLi impact?

EXPERIMENT - 10

Create a Simulated Phishing Email and Track Which Users Respond

OBJECTIVE

To design and send a simulated (benign) phishing email within an authorised lab/training environment to raise security awareness, and to track which recipients interact with it, without capturing or misusing any real credentials or data.

TOOLS / SOFTWARE REQUIRED

- A lab/training email environment or a phishing-awareness platform (e.g., GoPhish, used strictly in an isolated/authorised setting)
- A test mail server or sandboxed mailbox group

THEORY

Phishing simulations are a controlled, defensive use of social-engineering techniques to measure and improve an organisation's human security posture. A simulated campaign sends a realistic but harmless email (e.g., mimicking a password-reset notice) to a consenting group of test recipients and records who opened the email or clicked the embedded tracking link, without collecting any real credentials. Results are used purely for awareness training, never for punitive action against individuals in this lab exercise.

PROCEDURE

1. Obtain explicit authorisation and define the scope: a closed list of consenting lab/test email addresses only.
2. Design a simulated phishing email template (e.g., a fake 'password expiry' notice) using generic, non-deceptive lab branding.
3. Set up a benign tracking link (e.g., a GoPhish campaign landing page) that simply logs a click event and displays an awareness message — no credential harvesting form is used.
4. Configure the campaign in the phishing-simulation platform with the sender, subject, and body content.
5. Launch the campaign to the test recipient list.
6. Monitor the dashboard for delivery, open, and click-through statistics.
7. Compile a report summarising the click-through rate and use it to recommend targeted security-awareness training.
8. Immediately disclose to all participants that this was a training simulation and share the learning objective.

SAMPLE COMMANDS / OUTPUT

Sample (benign) subject line used in the simulation:
 "Action Required: Your Lab Portal Password Expires Today" – linking only to an awareness landing page, never a credential form.

PRECAUTIONS

- Never target real users, real email accounts, or real organisational data without explicit written authorisation from institutional authority (e.g., IT/Training department).
- Never harvest actual credentials, OTPs, or personal data as part of the simulation.
- Immediately debrief participants after the exercise; this is a training exercise, not a live social-engineering attack, and must never be run outside a sanctioned awareness programme.

VIVA QUESTIONS

1. What is phishing, and how does it exploit human rather than technical vulnerabilities?
2. What is the difference between phishing, spear-phishing, and whaling?
3. What metrics (open rate, click rate, report rate) are used to evaluate a phishing-awareness campaign?
4. What ethical and legal safeguards must be observed before running any phishing simulation?
5. What organisational controls (SPF/DKIM/DMARC, security-awareness training, email gateways) help defend against phishing?

EXPERIMENT - 11

Assess and Report Physical Security Vulnerabilities in the Lab/Office

OBJECTIVE

To conduct a walkthrough physical-security assessment of the lab/office premises, identify vulnerabilities such as unattended systems, insecure access, or exposed sensitive information, and prepare a findings report.

TOOLS / SOFTWARE REQUIRED

- Physical security assessment checklist
- Notebook/camera for documentation (with permission)

THEORY

Physical security is a foundational pillar of information security — an attacker with physical access to a system can often bypass logical controls entirely (e.g., booting from removable media, plugging in a rogue USB device, or directly viewing unattended screens). A physical security assessment systematically reviews access controls, asset handling, and environmental safeguards to identify such risks before they are exploited.

PROCEDURE

1. Obtain permission from the lab in-charge/faculty to conduct the walkthrough assessment.
2. Prepare a checklist covering: entry/exit access control, visitor logging, CCTV coverage, unattended/unlocked workstations, exposed passwords (e.g., sticky notes), unsecured server/network cabinets, availability of fire-safety and backup power equipment, and disposal practices for printed material.
3. Walk through the lab/office and record observations against each checklist item.
4. Rate each finding by risk level (High/Medium/Low) based on likelihood and potential impact.
5. Photograph evidence only where permitted and appropriate (e.g., an unlocked server rack), respecting privacy.
6. Compile the findings into a physical-security assessment report with recommendations, such as auto-lock screen policies, cabinet locks, visitor registers, and clean-desk policy enforcement.
7. Present the report to the lab in-charge/faculty for review.

SAMPLE COMMANDS / OUTPUT

Example checklist entry:

Finding: Server rack in lab left unlocked | Risk: High | Recommendation: Install a keyed lock and restrict access to authorised staff only.

PRECAUTIONS

- Conduct this assessment only within the institute's own premises with prior permission — never assess third-party premises without authorisation.
- Do not photograph or record any individual without consent; focus on physical assets and processes.
- Report findings responsibly to the lab in-charge rather than publicising security gaps.

VIVA QUESTIONS

1. Why is physical security considered a foundational layer in an organisation's overall security posture?
2. Give three examples of physical security controls and the risks they mitigate.
3. What is 'tailgating' in the context of physical security?
4. How does an unattended, unlocked workstation constitute a security risk even if the network is otherwise secure?

5. What is a clean-desk policy, and why is it recommended?

EXPERIMENT - 12

Conduct a Mock Test to Find Sensitive Data from Discarded Documents (Safe Simulation)

OBJECTIVE

To conduct a controlled, safe simulation of 'dumpster diving' using pre-prepared mock discarded documents, in order to demonstrate how sensitive information can be recovered from improperly disposed paperwork, and to recommend secure disposal practices.

TOOLS / SOFTWARE REQUIRED

- A set of intentionally prepared mock documents (dummy printouts containing simulated sensitive data, no real personal/institutional data)
- Checklist for classifying recovered information

THEORY

Dumpster diving is a social-engineering/physical-reconnaissance technique where an attacker searches discarded trash for sensitive information such as printed passwords, network diagrams, internal memos, or personally identifiable information. Even in the digital age, improperly disposed physical documents remain a real information-leakage vector. This experiment uses only safely fabricated, dummy documents prepared specifically for the exercise, never actual discarded institutional or personal records.

PROCEDURE

1. Prepare a set of mock 'discarded' documents containing simulated (fictitious) sensitive data — e.g., a fake internal memo, a dummy password list, a mock network diagram — clearly created for this lab exercise only.
2. Place the mock documents in a designated bin/tray to simulate a waste bin, with faculty supervision.
3. As the 'tester', sift through the mock discarded material and record what categories of sensitive information could be recovered (credentials, internal contacts, org charts, financial data, etc.).
4. Classify each recovered item by sensitivity level and assess the potential impact if it had been real data in an attacker's hands.
5. Compile a short report recommending secure disposal controls: cross-cut shredding, locked confidential-waste bins, and a documented data-disposal policy.
6. Dispose of the mock documents securely after the exercise (shredding) to model correct practice.

SAMPLE COMMANDS / OUTPUT

```
Example finding:  
Mock document: "Server_Backup_Password.txt printout" | Sensitivity: High |  
Recommendation: Shred all printed credentials/config data before disposal; avoid  
printing secrets altogether.
```

PRECAUTIONS

- Use only fabricated/dummy documents created specifically for this exercise — never use actual institutional, staff, or student records, discarded or otherwise.
- Do not perform this exercise on real waste bins/dumpsters, which may involve trespassing, hygiene risks, or exposure to genuine third-party personal data.
- Securely destroy the mock materials after the demonstration.

VIVA QUESTIONS

1. What is dumpster diving in the context of information security, and why is it still relevant today?

2. What types of sensitive information are commonly found in improperly discarded documents?
3. What is the recommended way to securely dispose of confidential physical documents?
4. How does this technique relate to social engineering and physical security assessments?
5. What organisational policy would you recommend to prevent information leakage through document disposal?

EXPERIMENT - 13

Capture and Analyze Unencrypted Network Traffic

OBJECTIVE

To capture live network traffic on a lab network using a packet-analysis tool and to analyse unencrypted protocols to demonstrate the risk of transmitting data in cleartext.

TOOLS / SOFTWARE REQUIRED

- Kali Linux
- Wireshark / tcpdump
- A lab network with an unencrypted service (e.g., HTTP, Telnet, or FTP running on a target VM)

THEORY

Protocols such as HTTP, FTP, and Telnet transmit data, including credentials, in cleartext, making them susceptible to eavesdropping by anyone with access to the network path (e.g., via ARP spoofing on a switched network, or directly on a hub/shared segment). Packet-capture tools such as Wireshark allow a tester to record and inspect raw network traffic frame by frame, demonstrating concretely why encrypted protocols (HTTPS, SFTP, SSH) are necessary.

PROCEDURE

1. Identify the correct network interface for capture: ip a or Wireshark's capture interface list.
2. Start a capture on the lab network interface using Wireshark (or tcpdump -i eth0 -w capture.pcap).
3. On the target VM, perform an action using an unencrypted protocol, e.g., log in via FTP or Telnet, or submit a plaintext HTTP login form.
4. Stop the capture once the transaction is complete.
5. In Wireshark, apply a display filter for the relevant protocol, e.g., ftp, telnet, or http.
6. Use 'Follow TCP Stream' on the relevant packet to reconstruct the full conversation and locate the transmitted username/password in cleartext.
7. Document the captured credentials/data as evidence (redacted appropriately in the report) and recommend switching to encrypted equivalents (HTTPS, SSH/SFTP).

SAMPLE COMMANDS / OUTPUT

```
tcpdump -i eth0 -w capture.pcap  
Wireshark filter: ftp || telnet || http.request
```

PRECAUTIONS

- Capture traffic only on the isolated lab network segment you are authorised to monitor — intercepting third-party network traffic without consent is illegal under the IT Act, 2000.
- Use only lab test accounts/credentials for the demonstration, never real personal accounts.
- Securely delete capture files containing any credential material after the exercise is documented.

VIVA QUESTIONS

1. Why are protocols like Telnet and FTP considered insecure compared to SSH and SFTP?
2. What is packet sniffing, and under what network conditions is it easiest to perform (hub vs switch)?
3. What is ARP spoofing, and how does it enable traffic interception on a switched network?
4. How does Wireshark's 'Follow TCP Stream' feature assist in traffic analysis?

5. What encryption-based countermeasures prevent cleartext credential capture?

EXPERIMENT - 14

Use SSL Labs or OpenSSL to Test the Security of a Web Server's SSL Configuration

OBJECTIVE

To assess the SSL/TLS configuration of a web server using Qualys SSL Labs (for a public-facing test site) and the openssl command-line tool (for a lab-hosted server), and to identify weak protocols, ciphers, or certificate issues.

TOOLS / SOFTWARE REQUIRED

- Web browser (for SSL Labs)
- Kali Linux with openssl
- A lab web server configured with TLS (e.g., Apache/Nginx with a self-signed or test certificate)

THEORY

TLS (formerly SSL) secures web traffic through certificate-based authentication and encrypted channels. Misconfigurations — support for deprecated protocols (SSLv3, TLS 1.0/1.1), weak cipher suites, self-signed or expired certificates, or missing HSTS — weaken this protection. Tools such as Qualys SSL Labs' 'SSL Server Test' provide an automated, graded (A–F) assessment for internet-facing servers, while openssl s_client allows direct command-line inspection of a server's certificate and negotiated protocol/cipher, which is essential for testing internal/lab servers not reachable from the internet.

PROCEDURE

1. For a public test domain you are authorised to assess: open <https://www.ssllabs.com/ssltest/> in a browser and enter the hostname; review the summary grade, protocol support, cipher suites, and certificate chain details in the report.
2. For a lab-hosted server, use openssl to inspect the certificate: `openssl s_client -connect <lab-server>:443 -servername <lab-server>`.
3. Extract and review the certificate details (issuer, validity dates, subject) printed by s_client, or pipe through `openssl x509 -noout -text`.
4. Test which protocol versions are accepted, e.g.: `openssl s_client -connect <lab-server>:443 -tls1` to check if the deprecated TLS 1.0 is still enabled.
5. List the cipher suites offered by the server using a tool such as nmap `--script ssl-enum-ciphers -p 443 <lab-server>`.
6. Compile findings: supported protocol versions, cipher strength, certificate validity/self-signed status, and presence/absence of HSTS.
7. Recommend remediations: disable SSLv3/TLS1.0/1.1, enforce strong cipher suites, use a valid CA-signed certificate, and enable HSTS.

SAMPLE COMMANDS / OUTPUT

```
openssl s_client -connect labserver.local:443 -servername labserver.local
nmap --script ssl-enum-ciphers -p 443 labserver.local
```

PRECAUTIONS

- Only run the SSL Labs public test against a domain you own/administer, as the test results are cached and publicly viewable by default (use the 'Do not show the results on the boards' option).
- Test openssl/nmap SSL enumeration only against the lab server, not external production systems, without authorisation.

- Treat certificate private keys as strictly confidential; never expose them during the exercise.

VIVA QUESTIONS

1. What is the difference between SSL and TLS, and which versions are considered deprecated/insecure today?
2. What does a certificate chain consist of, and why does an incomplete chain cause browser warnings?
3. What is HSTS, and what attack does it help prevent?
4. What is a weak cipher suite, and why should it be disabled on a production server?
5. How does openssl s_client differ from using a browser-based tool like SSL Labs for TLS testing?

BEYOND THE SYLLABUS

Beyond the Syllabus Exp - 1: End-to-End VAPT Report of a Vulnerable Web Application

Objective: To integrate the skills learnt across all experiments — reconnaissance, scanning, vulnerability assessment, exploitation, and traffic analysis — by conducting a complete mini VAPT engagement on a lab-hosted vulnerable web application (e.g., DVWA), and to compile a professional penetration-testing report covering scope, methodology, findings (with CVSS ratings), evidence, and remediation recommendations.

Beyond the Syllabus Exp - 2: Building a Secure Configuration Baseline

Objective: To take a deliberately misconfigured lab server (weak SSH config, outdated services, missing security headers) and apply hardening measures — disabling weak protocols/ciphers, applying security headers, enforcing key-based SSH authentication, and patching outdated services — then re-run the scanning tools from earlier experiments to verify the improvement in security posture.